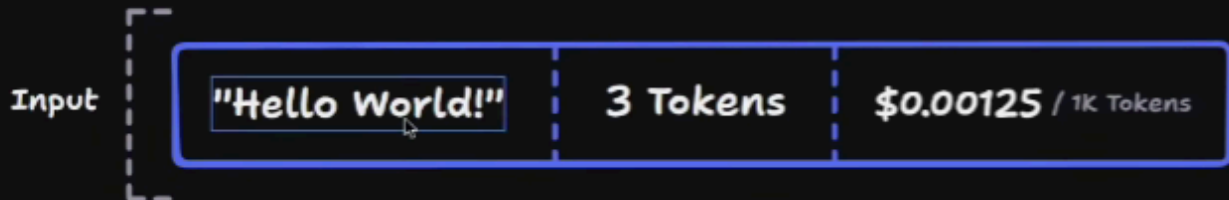


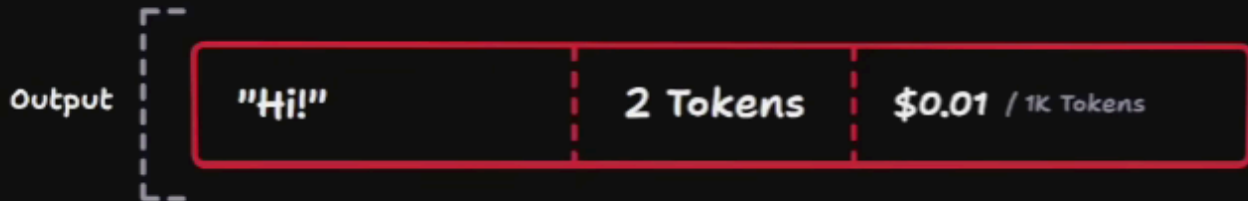
Tokens

Charged By Token

Tokens are the currency of LLM's



Thinking...



Tokens - Tokens are **the basic units of text** that a large language model (LLM) processes, representing words, parts of words, or punctuation marks.

Encoding

Turning text into tokens

"Hello World!"

Split

"Hello "

"World"

!"

Encode

13225

5922

0

Decoding

Turning tokens into text

13225

5922

0

Decode

"Hello "

"World"

!"

Join

"Hello World!"

LLM Process

The full LLM process, end-to-end

"Hello World!"

Split

"Hello "

"World"

!"

Encode

13225

5922

0

LLM

Thinking...

Output

12194

0

Decode

"Hi"

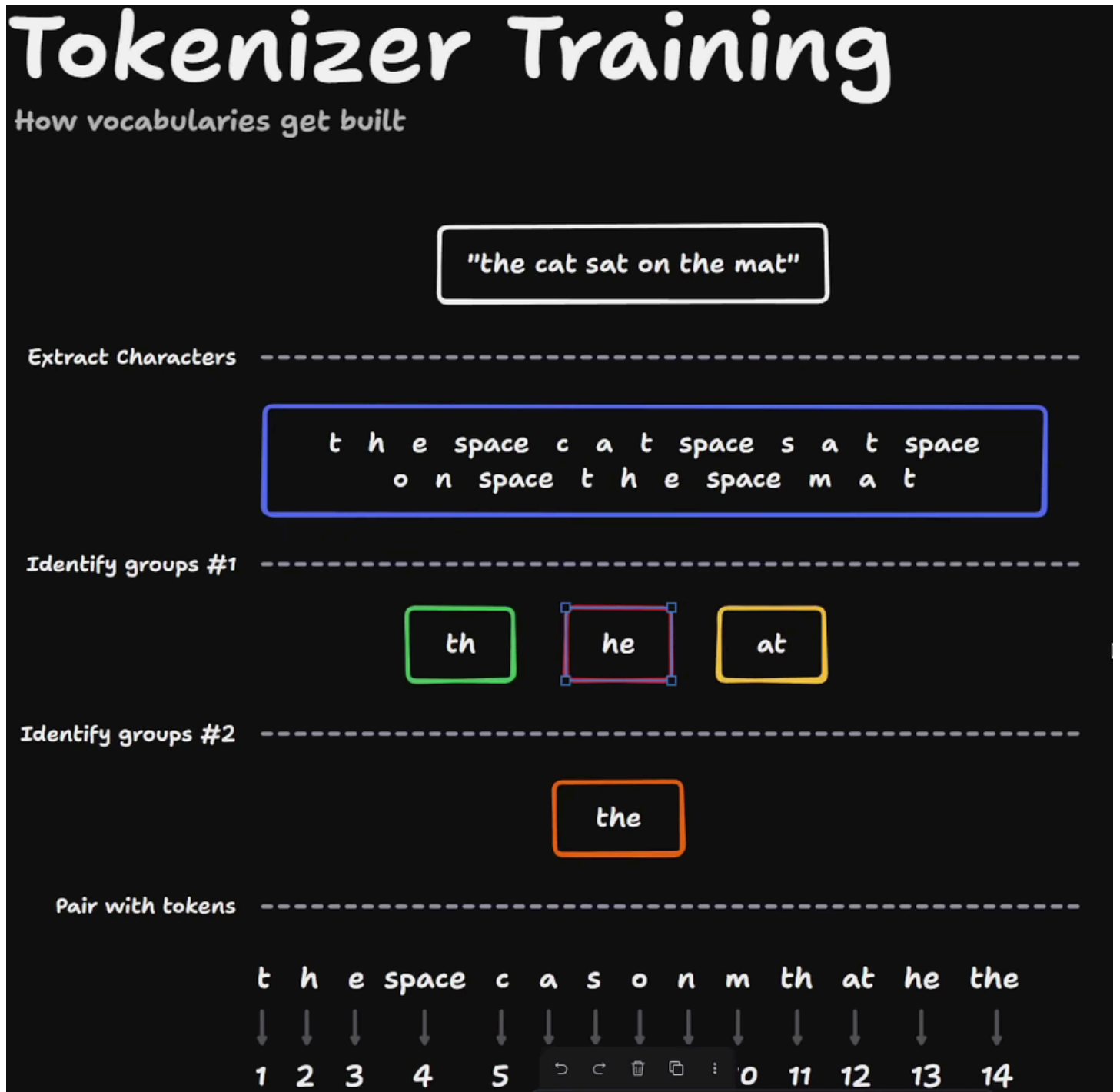
!"

Join

"Hi!"

Q.) Why tokens differ between model providers ?

Different model providers treat the input differently and produce different number of tokens based on the training as below.



Q.) How a tokenizer behaves when it encounters an unusual words?

LLM Tokenizer Unknown Word Rule:

1. Try matching the whole word.
2. If unknown, split into largest known subwords (BPE behavior).
3. If still unknown, fall back to character-level or byte-level tokens.
4. This ensures every possible text can be tokenized, but increases token count and reduces model understanding for rare words.

Practical Real Example

Take the fictional word:

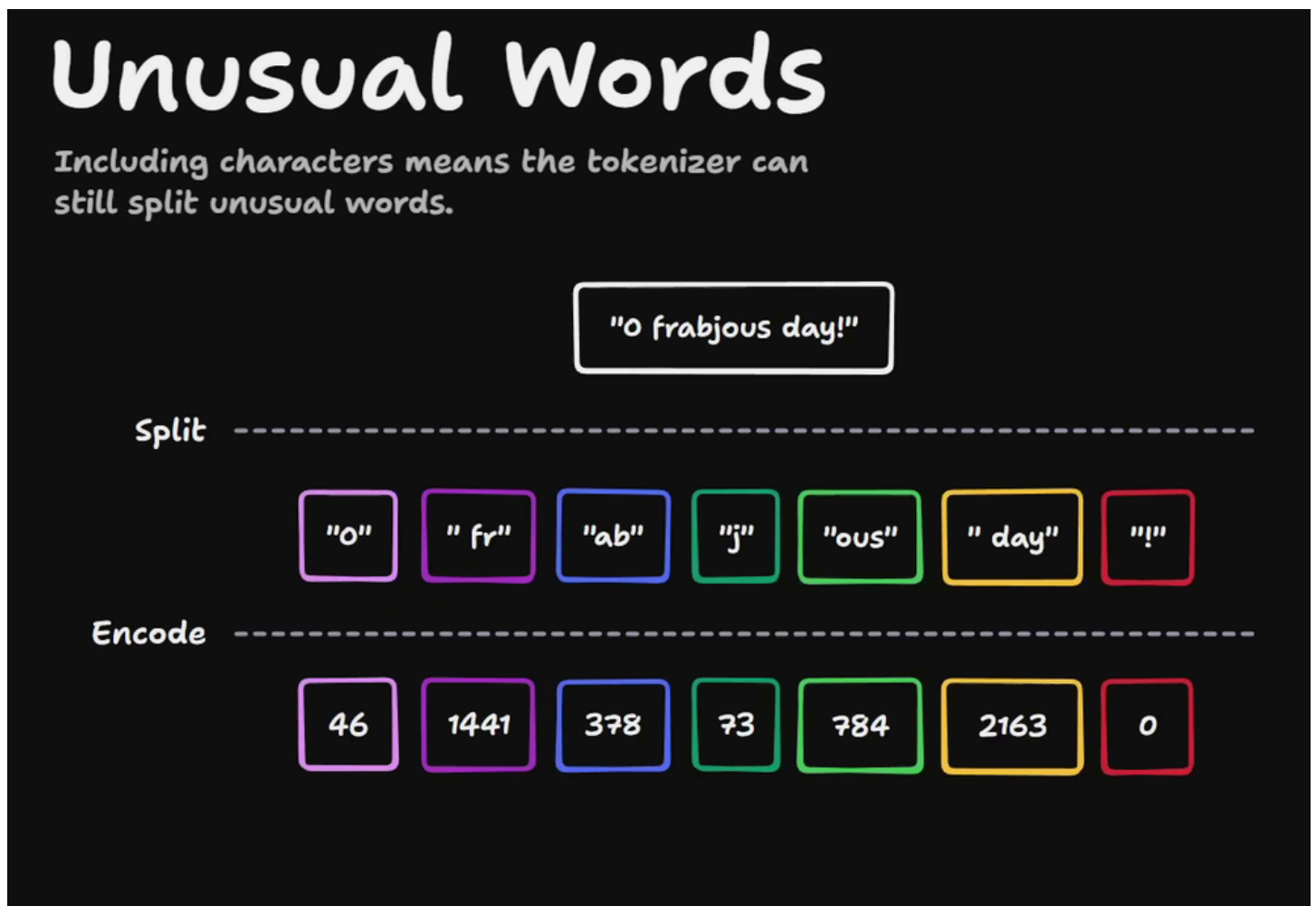
"glorpnacular"

GPT tokenizer might do:

["gl", "or", "p", "na", "cu", "lar"]

6 tokens for 1 word

(English typical word = 1-2 tokens)



Large Language Models (LLMs) cannot understand raw text directly; they require numbers. To bridge this gap, every LLM uses a unique **Token Vocabulary**—a database of all the words, parts of words (subwords), and characters it recognizes.

When you send a prompt to an LLM, the following happens:

1. **Segmentation:** The text is chopped up into the largest possible chunks (tokens) found in its vocabulary.
2. **Encoding:** Each chunk is swapped for its specific identification number (Token ID).
3. **Processing:** The LLM processes this list of numbers to generate a response.
 - **Subword Tokenization (Efficiency):**

- LLMs don't just store whole words. If they did, their vocabulary would be infinite. Instead, they use "subwords."
 - *Common word*: "apple" 1 token.
 - *Complex/Rare word*: "antidisestablishmentarianism" broken into multiple sub-tokens (e.g., anti, dis, establish, etc.).
 - This allows the model to understand words it has never seen before by combining parts it *does* know.
- **Spaces Matter:**
- Tokenizers handle spaces differently. In many models (like GPT-4), a space is often part of the word that follows it.
 - "Hello world" might be tokenized as ["Hello", " world"] (notice the space included in the second token).
- **The "Context Window":**
- LLMs have a limit on how much information they can remember at once, known as the context window. This limit is measured in tokens, not words.
- *Rule of Thumb*: 1,000 tokens is roughly 750 words.

LLM stops generating text when it predicts a special END token, reaches a max token limit, hits a stop sequence, or when its probability model decides that any further continuation would be unnatural based on training patterns.

Q.) Since LLM don't store whole word but store sub words so where do they store sub words in some database and since there are so many infinite words possible and even sub-words so how are they stored?

Ans by LLM and internet research -

You have hit on the exact problem that computer scientists had to solve: **Language is infinite, but computer memory is finite.**

Here is the answer to your question: They do **not** store an infinite number of subwords. They store a **fixed, finite list** in a simple file.

Here is exactly where and how they are stored.

1. Where is it stored? (The "Database")

It is not a complex SQL database or a cloud server. It is usually just a simple text or JSON file that sits right next to the model file on the hard drive.

If you were to download a model (like Llama 3 or GPT-2) to your computer, you would see a file often named vocab.json or tokenizer.model.

It looks like a giant two-column spreadsheet (a Lookup Table):

Token String	Token ID
"a"	1
"b"	2
...	...
"ing"	256
"the"	257
"apple"	1500
"antidis"	4500

The Size Limit:

Every LLM has a hard limit on this file size.

- **GPT-4:** approx. 100,000 unique tokens.
- **Llama 3:** approx. 128,000 unique tokens.

The model strictly cannot "know" any token that is not in this specific list of 100k–128k items.

2. How do they handle "Infinite" words with a fixed list?

Think of this like **The Periodic Table of Elements**.

There are an infinite number of molecules in the universe (water, plastic, DNA, stars). However, there are only about 118 distinct elements (Hydrogen, Oxygen, Carbon, etc.).

- You don't need a unique entry for "Water" if you have "Hydrogen" and "Oxygen."

The LLM Vocabulary works the same way:

1. **The Base:** The vocabulary *always* contains every individual character (a, b, c, ..., z) and symbol. This ensures that **no word is impossible to write**.
2. **The Combinations:** It contains the most common "molecules" (subwords) found in the internet data it was trained on.

Example: The word "Uninstagrammable"

If this word is not in the vocabulary, the tokenizer breaks it down until it finds pieces that *are* in the list:

1. Check for "Uninstagrammable" *Not found*.
2. Check for "Uninstagram" *Not found*.
3. Check for "Un" **Found!** (Token ID 55)
4. Check for "instagram" **Found!** (Token ID 889)
5. Check for "mable" *Not found*.
6. Check for "mab" *Not found*.
7. Check for "m" **Found!**
8. Check for "able" **Found!**

Result: [55, 889, 12, 403]

3. How was the list chosen? (Byte Pair Encoding) also refer to tokenizer training pic

You asked how the subwords are stored since there are so many possibilities. Before the AI is ever trained to "speak," a computer program reads terabytes of text and plays a survival game called **Byte Pair Encoding (BPE)**:

1. **Start small:** The list starts with *only* individual characters (a, b, c...).
2. **Count frequency:** The program scans the internet text and notices that t and h appear together constantly.
3. **Merge:** It creates a new token th.
4. **Repeat:** It notices th and e appear together constantly. It creates the.
5. **Stop:** It keeps doing this, merging frequent pairs into larger subwords, until it hits the limit (e.g., 100,000 tokens).

Rare words (like your name or a made-up word) never appear frequently enough to get their own "merged" token, so they get stuck being represented by smaller chunks or individual letters.

The Takeaway

The "database" is just a **dictionary file**.

It solves the "infinite words" problem by having a **safety net**: if a word is too complex or rare to have its own token ID, the model simply spells it out using the tokens for individual syllables or letters.

Architecture: The Tokenizer vs. The LLM

1. The Core Question

Who performs the tokenization process?

Answer: The tokenization is **not** performed by the LLM (the Neural Network). It is performed by a separate, lightweight software component called **The Tokenizer**.

2. The Workflow (Step-by-Step)

The process is a relay race between two distinct pieces of software running on different hardware.

1. **Input:** User provides raw text (e.g., "Hello").
2. **The Tokenizer (Preprocessing):**
 - **Action:** Breaks text into chunks and converts them to integers using the specific vocab.json lookup table.
 - **Hardware:** Runs on the **CPU**.
 - **Output:** A list of IDs (e.g., [15496]).
3. **The LLM (Processing):**
 - **Action:** Receives the IDs, performs complex matrix multiplication and logic.
 - **Hardware:** Runs on the **GPU** (Graphics Processing Unit).
 - **Output:** A list of probability IDs for the response (e.g., [995]).
4. **The Tokenizer (Post-processing):**
 - **Action:** Converts the output IDs back into human text.
 - **Output:** Final text response (e.g., "World").

Key Takeaway: The LLM never sees "text." It only ever receives and outputs lists of numbers. The Tokenizer acts as the bridge.

3. The "Married Couple" Rule

The Tokenizer and the LLM are strictly coupled.

- **Relationship:** Every model is trained on a specific vocabulary list. If the model learns that ID 5 = "Hello", you must use the specific tokenizer that maps "Hello" to 5.
- **Incompatibility:** You **cannot** use OpenAI's tokenizer with Meta's Llama model. They use different "codebooks."
 - *GPT-4 Tokenizer:* "Hello" ID 500
 - *Llama-3 LLM:* Interprets ID 500 as "Sandwich" (Hypothetically).
- **Exception (Model Families):** "Sibling" models usually share a tokenizer.
 - *Example:* Llama-3 (8 Billion) and Llama-3 (70 Billion) share the same tokenizer file.

4. Why are they separate? (Optimization)

- **The LLM:** Optimized for **Math** (Floating point operations, Probability). It is computationally expensive and slow.
- **The Tokenizer:** Optimized for **String Manipulation** (Text matching, RegEx). It is computationally cheap and fast.
- *Reason:* Separating them allows the heavy "Brain" (LLM) to focus purely on logic without wasting resources on basic text formatting.

5. Conceptual Analogy

- **The LLM (The King):** A brilliant mathematician who sits in a fortress. He solves complex problems but **only understands numbers**. He cannot read or speak human languages.
- **The Tokenizer (The Translator):** A fast messenger at the gate. He cannot solve complex problems, but he has a book that translates **Words Numbers**.
 - *The King cannot function without the Translator.*

Q.) As english symbols are stored with token id and since models answer in so many language like hindi, spanish etc so in that same token limit of symbols and sub-words these are also stored?

1. One Vocabulary for the Entire Model

- Every LLM (GPT, Llama, etc.) has **one shared vocabulary list** (≈100k–128k tokens).
- This single list must store patterns for:
 - English
 - Spanish, French
 - Hindi, Bengali, Tamil
 - Chinese, Japanese
 - Code (Python, JavaScript, etc.)
 - Emojis
- All languages share the same “token real estate.”

2. Unicode (UTF-8) as the Base

- Computers store text using **Unicode**, covering all world languages.
- Tokenizer doesn't look for just English letters — it analyzes **byte patterns**.

In other words, When the tokenizer is built, it doesn't just look for "a, b, c." It looks for **any** frequent pattern in the training data bytes.

- If the training data has a lot of Spanish, the tokenizer learns that hola is frequent and assigns it one token ID.
- If the training data has a lot of Hindi, it learns that है is frequent and assigns it one token ID.

3. The “Real Estate Problem” of Tokenizers

- Most training data = **majority English (often 80%+)**.
- Tokenizer merges based on frequency.
- Therefore:

Language	Frequency in training	Token Efficiency
English	Extremely high	Whole words = 1 token
Spanish/French	High	Many whole words
Hindi/Indic languages	Lower	Split into many sub-tokens
Rare characters	Very low	Split into byte tokens

4. The “Token Tax” on Hindi & Similar Languages

The same sentence costs more tokens in Hindi than in English.

Example:

English:

“I am going home.”

→ ~5 tokens

Hindi:

“मैं घर जा रहा हूँ”

→ 10–15 tokens (depends on model)

Reason:

Hindi words are often broken into small parts like:

["म", "ैं", " घ", "र", " ज", "ा", ...]

5. How the Vocabulary Stores Languages

A conceptual view of vocab.json:

Token ID	Token	Notes
101	"the"	English
102	"and"	English
1540	"la"	Spanish
3050	"न"	Hindi character
3051	"म"	Hindi character
3052	"नमस्ते"	Hindi whole word (rare)
9055	"😊"	Emoji

Hindi gets:

- Fewer whole-word tokens
- More single-character tokens
- More fragmentation

6. What if a Token Does NOT Exist? (Fallback to Bytes)

If a character is too rare to be in the vocab list:

- Model falls back to **RAW BYTES**, e.g.,
 - One Devanagari character might become **3 or 4 tokens**.
- This guarantees the model can still read/write any language, but very inefficiently.

7. Vocabulary Structure (Diagram)

Structure of the 100k Token Vocabulary

"English Words & Subwords" : 60

"Code (Python, HTML, etc.)" : 15

"European Langs (Spanish, French)" : 15

"Asian/Indic Scripts (Hindi, Kanji)" : 5

"Symbols, Whitespace, Emojis" : 5

8. Consequences of a Small Share for Hindi

Because Hindi & Indic languages get a tiny slice of the vocabulary:

1. More Tokens Needed

Same meaning → 2× or 3× tokens vs English.

2. Slower Generation

Model must generate more tokens → slower output.

3. Smaller Context Window

Because tokens fill faster → long Hindi text = quicker overflow.

4. Higher Cost

If API charges per token, Hindi literally becomes more expensive.

Final Summary

LLM tokenizers allocate vocabulary based on training data frequency.

Since English dominates, it gets most whole-word tokens.

Hindi and similar scripts get fewer, leading to fragmented tokens, slower generation, faster context usage, and higher cost.

Token Usage: 6279 / 1048576

Input tokens: 513

Output tokens: 5,766

Total tokens: 6,279

Cost Estimation *

Input token cost: \$0.001026

Output token cost: \$0.069192

Total cost: \$0.070218

*: This is an estimated cost if you
make the same request via API.